StringBuffer Insert and Append

In this chapter you will learn:

1. How to append to StringBuffer
2. How to use the chained append() method
3. How to add data in the middle of a StringBuffer

Append to StringBuffer

The `append()` method appends the string to the end. It is overloaded to have the following forms. We can get usage for each method by the parameters. `append(boolean b)` is to append `boolean` value to the `StringBuffer`.

- StringBuffer append(boolean b)
- StringBuffer append(char c)
- StringBuffer append(char[] str)
- StringBuffer append(char[] str, int offset, int len)
- StringBuffer append(CharSequence s)
- StringBuffer append(CharSequence s, int start, int end)
- StringBuffer append(double d)
- StringBuffer append(float f)
- StringBuffer append(int i)
- StringBuffer append(long lng)
- StringBuffer append(Object obj)
- StringBuffer append(String str)
- StringBuffer append(StringBuffer sb)
- StringBuffer appendCodePoint(int codePoint)

The following code shows how to use the various `append` methods.

```
public class Main {
  public static void main(String[] argv) {
    StringBuffer sb = new StringBuffer();
    sb.append(true);//from   j a  va2s. c  o  m
    sb.append("java2s.com");
    sb.append(1.2);
    sb.append('a');
    System.out.println(sb.toString());
  }
}
```

The output:

```
truejava2s.com1.2a
```


`String.valueOf( )` is called for each parameter to obtain its string representation.

Chained append() method

The buffer itself is returned by each version of `append()`. This allows subsequent calls to be chained together, as shown in the following example:

```
public class Main {
  public static void main(String args[]) {
    String s;/* ja  v  a 2 s .  c om*/
    int a = 42;
    StringBuffer sb = new StringBuffer(40);
    s = sb.append("a = ").append(a).append("!").toString();
    System.out.println(s);
  }
}
```

The output of this example is shown here:

```
a = 42!
```


Insert to a StringBuffer

The `insert( )` method inserts one string into another. It is overloaded to accept values of all the simple types, plus `Strings`, Objects, and CharSequences. It calls `String.valueOf( )` to obtain the string representation of the value it is called with.

- StringBuffer insert(int offset, boolean b)
- StringBuffer insert(int offset, char c)
- StringBuffer insert(int offset, char[] str)
- StringBuffer insert(int index, char[] str, int offset, int len)
- StringBuffer insert(int dstOffset, CharSequence s)
- StringBuffer insert(int dstOffset, CharSequence s, int start, int end)
- StringBuffer insert(int offset, double d)
- StringBuffer insert(int offset, float f)
- StringBuffer insert(int offset, int i)
- StringBuffer insert(int offset, long l)
- StringBuffer insert(int offset, Object obj)
- StringBuffer insert(int offset, String str)

The following sample program inserts strings:

```
public class Main {
  public static void main(String[] argv) {
    StringBuffer sb = new StringBuffer();
    sb.append(true);/*from  j ava 2s. c  o  m*/
    sb.append("java2s.com");
    sb.append(1.2);
    sb.append('a');

    sb.insert(3,"JAVA@S.COM");
    System.out.println(sb.toString());
  }
}
```

The output:

```
truJAVA@S.COMejava2s.com1.2a
```


Next chapter...

What you will learn in the next chapter:

1. What is difference between length and capacity of a Java StringBuffer
2. How to Pre-allocate space
3. How to shorten a StringBuffer

« Previous Next »